CMSC132: Graphs in Depth

Chapter 17

By Naren Ayinala

May 2026

Contents

1 Why Do We Need Advanced Graph Algorithms? 3

2 Weighted Graphs and Graph Representations 3

2.1 Adjacency List Representation 3

2.2 Adjacency Matrix Representation 4

3 Shortest Paths 4

3.1 Dijkstra's Algorithm 4

3.2 Dijkstra Worked Example 4

3.3 Dijkstra in Java 5

3.4 Why Dijkstra Does Not Work with Negative Edges 6

4 Bellman Ford Algorithm 6

5 Minimum Spanning Trees 7

5.1 Kruskal's Algorithm 8

5.2 Union Find and Path Compression 8

5.3 Kruskal in Java 8

5.4 Prim's Algorithm 9

6 Topological Sorting 10

6.1 Kahn's Algorithm Using Indegrees 11

6.2 Topological Sort Using DFS 11

7 Connected Components and Strongly Connected Components 12

8 Detecting Cycles 13

9 Which Algorithm Should We Use? 13

10 Common Mistakes to Avoid **13**

11 Practice Questions **14**

11.1 Answers 14

1 Why Do We Need Advanced Graph Algorithms?

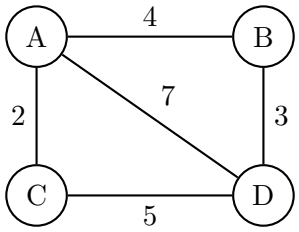
In the introduction to graphs, we learned that a graph is made of vertices and edges, and that BFS and DFS let us explore which vertices are reachable. However, simply knowing that we can reach a vertex is not always enough. What if every road has a different travel time? What if we want to connect several cities while building as few roads as possible? What if certain tasks must finish before other tasks can begin?

These are the situations where advanced graph algorithms become useful. In this chapter, we will build on adjacency lists, BFS, and DFS to study shortest paths, minimum spanning trees, topological sorting, and related graph problems. We will also focus on how to implement them in Java and why each algorithm works.

Source note: The uploaded CMSC132 notes contain the heading "Graphs in Depth" without developed notes underneath it. The explanations, examples, diagrams, and Java code in this chapter are newly added study material, using the same explanatory style as the rest of the notes.

2 Weighted Graphs and Graph Representations

A weighted graph is a graph where every edge has an associated value called its **weight**. This value could represent distance, cost, time, or any other quantity. For example, an edge with weight 4 could mean that traveling between two cities takes four minutes.



Notice that the weight is not necessarily the same as the physical length of the line in our drawing. The number written next to the edge is what matters.

2.1 Adjacency List Representation

For an unweighted graph, we can store a list of neighbors for every vertex. For a weighted graph, we also need to store the weight of the connection. A useful way is to create an `Edge` class.

```
import java.util.*;

class Edge {
    int to;
    int weight;

    Edge(int to, int weight) {
        this.to = to;
        this.weight = weight;
    }
}

List<List<Edge>> graph = new ArrayList<>();
for (int i = 0; i < 4; i++) {
    graph.add(new ArrayList<>());
}
// Directed edge: 0 -> 1 with weight 4
// graph.get(0).add(new Edge(1, 4));
```

The list at index `0` stores all edges leaving vertex 0. For an undirected graph, remember to add both directions. For example, if A connects to B with weight 4, store A to B and B to A.

2.2 Adjacency Matrix Representation

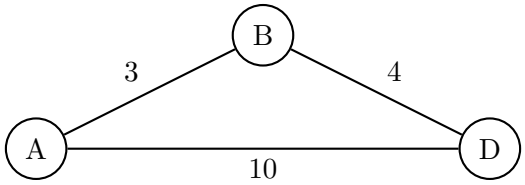
An adjacency matrix uses a two dimensional array. The entry $matrix[i][j]$ represents the weight of the edge from vertex i to vertex j. We need a special value for a missing edge, often a very large number called ∞. The diagonal normally has zero because the distance from a vertex to itself is zero.

	A	B	C	D
A	0	4	2	7
B	4	0	∞	3
C	2	∞	0	5
D	7	3	5	0

An adjacency list uses $O(V + E)$ memory, while an adjacency matrix uses $O(V^2)$. A matrix makes it easy to check the weight of a particular edge in $O(1)$, while a list is usually better when relatively few edges exist.

3 Shortest Paths

A shortest path is the path between two vertices with the smallest **total edge weight**. If A connects to D directly with weight 10, but A to B has weight 3 and B to D has weight 4, then going through B has total cost 7 and is better than the direct edge.



Important: BFS gives shortest paths in an unweighted graph (or when all edges have the same weight), but it does not generally give minimum total weight when edges have different costs.

3.1 Dijkstra's Algorithm

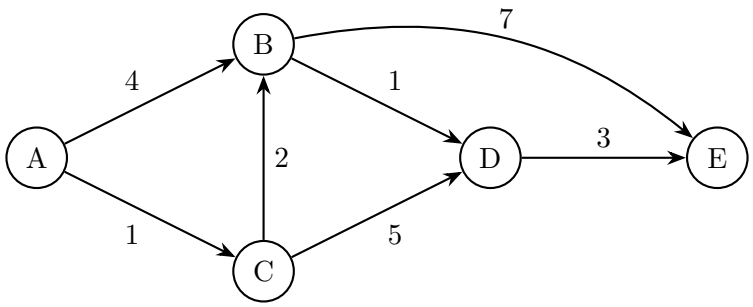
Dijkstra's algorithm finds the shortest distances from one starting vertex to every other vertex in a graph whose edge weights are **nonnegative**. The main idea is to repeatedly process the vertex with the smallest currently known distance and use its outgoing edges to improve distances to neighbors. This improvement is called **relaxation**.

If we currently know that the distance to A is 4 and an edge from A to B has weight 3, we have discovered a path to B with cost $4 + 3 = 7$. If the previous distance to B was 12, we replace 12 with 7.

$$\text{if } dist[u] + w(u, v) < dist[v], \quad dist[v] = dist[u] + w(u, v).$$

3.2 Dijkstra Worked Example

Consider the directed weighted graph below, starting from A.



At first, only A has distance 0. Every other distance is infinity. From A, we discover B with cost 4 and C with cost 1. Since C has the smaller cost, we process C next. Through C we can reach B with cost $1 + 2 = 3$, so we update B from 4 to 3. Eventually, D is reached with cost 4 and E with cost 7.

After processing	A	B	C	D	E
Initialization	0	∞	∞	∞	∞
A	0	4	1	∞	∞
C	0	3	1	6	∞
B	0	3	1	4	10
D	0	3	1	4	7
E	0	3	1	4	7

The shortest path from A to E is A, C, B, D, E with total weight $1 + 2 + 1 + 3 = 7$.

3.3 Dijkstra in Java

We can use Java's `PriorityQueue` to efficiently retrieve the next vertex with the smallest tentative distance. The queue stores pairs containing a vertex and its current distance. We might insert the same vertex more than once, so we skip outdated entries when we remove them from the queue.

```
import java.util.*;

public class DijkstraExample {
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to;
            this.weight = weight;
        }
    }

    static class State {
        int node;
        long distance;
        State(int node, long distance) {
            this.node = node;
            this.distance = distance;
        }
    }

    static long[] dijkstra(List<List<Edge>> graph, int start) {
        int n = graph.size();
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        dist[start] = 0;

        PriorityQueue<State> pq = new PriorityQueue<>(
            Comparator.comparingLong(s -> s.distance));
```

```
        pq.offer(new State(start, 0));

        while (!pq.isEmpty()) {
            State cur = pq.poll();
            if (cur.distance != dist[cur.node]) continue;

            for (Edge edge : graph.get(cur.node)) {
                long nextDist = cur.distance + edge.weight;
                if (nextDist < dist[edge.to]) {
                    dist[edge.to] = nextDist;
                    pq.offer(new State(edge.to, nextDist));
                }
            }
        }
        return dist;
    }

    public static void main(String[] args) {
        int n = 5;
        List<List<Edge>> g = new ArrayList<>();
        for (int i = 0; i < n; i++) g.add(new ArrayList<>());
        g.get(0).add(new Edge(1, 4));
        g.get(0).add(new Edge(2, 1));
        g.get(2).add(new Edge(1, 2));
        g.get(1).add(new Edge(3, 1));
        g.get(2).add(new Edge(3, 5));
        g.get(3).add(new Edge(4, 3));
        g.get(1).add(new Edge(4, 7));
        System.out.println(Arrays.toString(dijkstra(g, 0)));
        // [0, 3, 1, 4, 7]
    }
}
```

With adjacency lists and a binary heap priority queue, this implementation takes $O((V + E) \log V)$ time for a typical connected graph, or more generally $O((V + E) \log(V + E))$ with duplicate queue entries. Space is $O(V + E)$ for the graph and up to $O(E)$ queued entries.

3.4 Why Dijkstra Does Not Work with Negative Edges

Dijkstra relies on the fact that once the smallest distance is selected, no later path using nonnegative edges can improve it. Negative edges break that reasoning. A vertex that looks optimal now might later be reached more cheaply through a negative edge. Use Bellman Ford when negative edges are allowed.

4 Bellman Ford Algorithm

Bellman Ford also finds shortest distances from a starting vertex, but it supports negative edge weights. It repeatedly relaxes every edge, up to $V - 1$ passes. Why $V - 1$? A shortest simple path contains at most $V - 1$ edges. One additional pass detects a reachable negative cycle if any distance can still improve.

A **negative cycle** is a cycle whose edge weights sum to a negative number. If we can reach such a cycle, we can travel around it repeatedly and keep reducing the path cost. In that case, there is no finite shortest distance for vertices reachable from that cycle.

```
import java.util.*;

class BellmanFord {
    static class Edge {
```

```
        int from, to, weight;
        Edge(int from, int to, int weight) {
            this.from = from;
            this.to = to;
            this.weight = weight;
        }
    }

    static long[] shortestPaths(int n, List<Edge> edges,
                                int start) {
        long INF = Long.MAX_VALUE / 4;
        long[] dist = new long[n];
        Arrays.fill(dist, INF);
        dist[start] = 0;

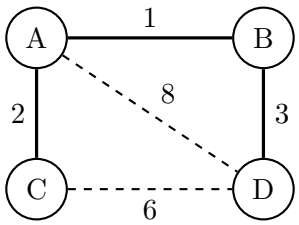
        for (int i = 1; i < n; i++) {
            boolean changed = false;
            for (Edge e : edges) {
                if (dist[e.from] != INF &&
                    dist[e.from] + e.weight < dist[e.to]) {
                    dist[e.to] = dist[e.from] + e.weight;
                    changed = true;
                }
            }
            if (!changed) break;
        }
        for (Edge e : edges) {
            if (dist[e.from] != INF &&
                dist[e.from] + e.weight < dist[e.to]) {
                throw new IllegalStateException(
                    "Reachable negative cycle");
            }
        }
        return dist;
    }
}
```

Time complexity is $O(VE)$, and extra space for the distance array is $O(V)$. It is slower than Dijkstra on most nonnegative weighted graphs, but it handles a more general problem.

5 Minimum Spanning Trees

A **spanning tree** of a connected undirected graph is a collection of edges that connects every vertex without creating any cycles. Since it is a tree, it has exactly $V - 1$ edges. A **minimum spanning tree**, or MST, is the spanning tree with the smallest possible total edge weight.

Imagine that we need to connect five buildings with cables. We want every building connected, but we want to minimize the total cable length. That is an MST problem. Notice that an MST is not the same as a shortest path tree: the MST minimizes the sum of all chosen edges, not the distance from a particular starting vertex.



In this example, the three solid edges connect all four vertices with total cost $1 + 2 + 3 = 6$. The

dashed edges are unnecessary and would make the total cost larger.

5.1 Kruskal's Algorithm

Kruskal's algorithm uses a greedy approach. It sorts all edges from smallest to largest and then adds an edge if doing so will not create a cycle. We continue until we have selected $V - 1$ edges. The most useful tool here is a **Disjoint Set Union** (DSU), also called Union Find, which keeps track of which vertices are already connected.

Example: Suppose our edges have weights AB=1, AC=2, BD=3, CD=6, AD=8. Kruskal selects AB, AC, and BD. At that point, all vertices are connected and we stop. The total is 6.

5.2 Union Find and Path Compression

Each vertex starts in its own set. `find(x)` tells us the representative of the set containing x. `union(a,b)` merges two sets if they are different. Path compression makes later `find` operations faster by connecting visited vertices directly to their representative. Union by size keeps the smaller set under the larger one.

```
class DSU {
    int[] parent, size;
    DSU(int n) {
        parent = new int[n];
        size = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            size[i] = 1;
        }
    }
    int find(int x) {
        if (parent[x] != x) parent[x] = find(parent[x]);
        return parent[x];
    }
    boolean union(int a, int b) {
        int rootA = find(a), rootB = find(b);
        if (rootA == rootB) return false;
        if (size[rootA] < size[rootB]) {
            int temp = rootA;
            rootA = rootB;
            rootB = temp;
        }
        parent[rootB] = rootA;
        size[rootA] += size[rootB];
        return true;
    }
}
```

5.3 Kruskal in Java

```
import java.util.*;

public class KruskalExample {
    static class Edge {
        int u, v, w;
        Edge(int u, int v, int w) {
            this.u = u; this.v = v; this.w = w;
        }
    }
    static class DSU {
```

```
        int[] parent, size;
        DSU(int n) {
            parent = new int[n]; size = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i; size[i] = 1;
            }
        }
        int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }
        boolean union(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (size[a] < size[b]) {
                int temp = a; a = b; b = temp;
            }
            parent[b] = a;
            size[a] += size[b];
            return true;
        }
    }
    static long mstWeight(int n, List<Edge> edges) {
        edges.sort(Comparator.comparingInt(e -> e.w));
        DSU dsu = new DSU(n);
        long total = 0;
        int used = 0;
        for (Edge e : edges) {
            if (dsu.union(e.u, e.v)) {
                total += e.w;
                used++;
                if (used == n - 1) break;
            }
        }
        if (used != n - 1) {
            throw new IllegalArgumentException(
                "Graph is disconnected");
        }
        return total;
    }
    public static void main(String[] args) {
        List<Edge> edges = new ArrayList<>();
        edges.add(new Edge(0, 1, 1));
        edges.add(new Edge(0, 2, 2));
        edges.add(new Edge(1, 3, 3));
        edges.add(new Edge(2, 3, 6));
        edges.add(new Edge(0, 3, 8));
        System.out.println(mstWeight(4, edges)); // 6
    }
}
```

Kruskal takes $O(E \log E)$ time because sorting the edges dominates. Union Find operations are almost constant time in practice with path compression and union by size.

5.4 Prim's Algorithm

Prim's algorithm is another way to build an MST. Instead of sorting all edges first, we start with one vertex and repeatedly choose the cheapest edge that connects a vertex already in our growing tree to a vertex outside it. A priority queue helps us select the cheapest next edge.

```
import java.util.*;

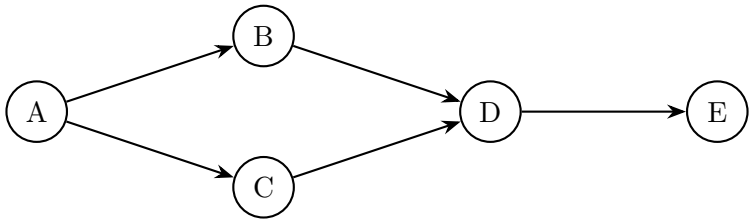
class PrimExample {
```

```
    static class Edge {
        int to, weight;
        Edge(int to, int weight) {
            this.to = to; this.weight = weight;
        }
    }
    static long prim(List<List<Edge>> graph) {
        int n = graph.size();
        if (n == 0) return 0;
        boolean[] visited = new boolean[n];
        // Each entry is [weight, vertex]
        PriorityQueue<int[]> pq = new PriorityQueue<>(
            Comparator.comparingInt(a -> a[0]));
        pq.offer(new int[]{0, 0});
        long total = 0;
        int count = 0;
        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int weight = cur[0], node = cur[1];
            if (visited[node]) continue;
            visited[node] = true;
            total += weight;
            count++;
            for (Edge e : graph.get(node)) {
                if (!visited[e.to]) {
                    pq.offer(new int[]{e.weight, e.to});
                }
            }
        }
        if (count != n) {
            throw new IllegalArgumentException(
                "Graph is disconnected");
        }
        return total;
    }
}
```

With an adjacency list and a priority queue, Prim runs in $O(E \log E)$ for this implementation with duplicate entries. For a connected graph this is often expressed as $O(E \log V)$ with suitable heap implementations.

6 Topological Sorting

Topological sorting is an ordering of the vertices of a **directed acyclic graph** (DAG) such that for every directed edge $u \to v$, u appears before v. Think of a course prerequisite system. If CMSC131 must be completed before CMSC132, then CMSC131 must appear first.



A valid topological order is A, B, C, D, E. Another valid order is A, C, B, D, E. Both work because B and C have no required order relative to each other. A graph containing a directed cycle cannot have a topological ordering.

6.1 Kahn's Algorithm Using Indegrees

The **indegree** of a vertex is the number of edges pointing into it. Kahn's algorithm starts by putting all vertices with indegree zero into a queue. We remove a vertex, add it to our answer, and reduce the indegree of its outgoing neighbors. Whenever a neighbor reaches indegree zero, we add it to the queue.

```
import java.util.*;

class TopologicalSort {
    static List<Integer> topoSort(List<List<Integer>> graph) {
        int n = graph.size();
        int[] indegree = new int[n];
        for (int u = 0; u < n; u++) {
            for (int v : graph.get(u)) indegree[v]++;
        }
        Queue<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) queue.offer(i);
        }
        List<Integer> order = new ArrayList<>();
        while (!queue.isEmpty()) {
            int u = queue.poll();
            order.add(u);
            for (int v : graph.get(u)) {
                indegree[v]--;
                if (indegree[v] == 0) queue.offer(v);
            }
        }
        if (order.size() != n) {
            throw new IllegalArgumentException(
                "Graph contains a directed cycle");
        }
        return order;
    }
}
```

The time complexity is $O(V + E)$, because each vertex enters the queue at most once and each edge is examined once.

6.2 Topological Sort Using DFS

We can also use DFS. After exploring all outgoing neighbors of a vertex, we add that vertex to a list. Reversing this finishing order gives a topological order, provided the graph has no directed cycle. For cycle detection, track three states: unvisited, visiting, and finished. Reaching a vertex that is currently visiting means we found a directed cycle.

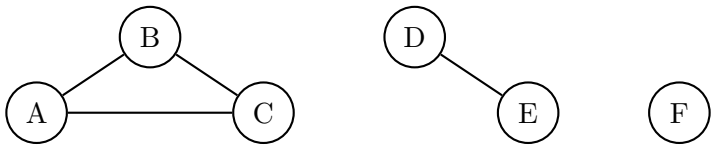
```
import java.util.*;

class DFSTopologicalSort {
    static void dfs(int u, List<List<Integer>> g,
                    int[] state, List<Integer> order) {
        state[u] = 1; // Currently visiting
        for (int v : g.get(u)) {
            if (state[v] == 1) {
                throw new IllegalArgumentException("Cycle");
            }
            if (state[v] == 0) dfs(v, g, state, order);
        }
        state[u] = 2; // Finished
        order.add(u);
    }
```

```
    static List<Integer> sort(List<List<Integer>> g) {
        int[] state = new int[g.size()];
        List<Integer> order = new ArrayList<>();
        for (int i = 0; i < g.size(); i++) {
            if (state[i] == 0) dfs(i, g, state, order);
        }
        Collections.reverse(order);
        return order;
    }
}
```

7 Connected Components and Strongly Connected Components

In an **undirected graph**, a connected component is a maximal group of vertices that can all reach one another. We can find components by running BFS or DFS from every vertex that has not been visited yet. Every new search discovers one component.



This graph has three components: $\{A, B, C\}$, $\{D, E\}$, and $\{F\}$. The isolated vertex F is itself a component.

```
import java.util.*;

class Components {
    static int countComponents(List<List<Integer>> graph) {
        boolean[] visited = new boolean[graph.size()];
        int count = 0;
        for (int start = 0; start < graph.size(); start++) {
            if (visited[start]) continue;
            count++;
            Queue<Integer> q = new ArrayDeque<>();
            q.offer(start);
            visited[start] = true;
            while (!q.isEmpty()) {
                int u = q.poll();
                for (int v : graph.get(u)) {
                    if (!visited[v]) {
                        visited[v] = true;
                        q.offer(v);
                    }
                }
            }
        }
        return count;
    }
}
```

For a directed graph, a **strongly connected component** (SCC) is a maximal group of vertices where every vertex can reach every other vertex by following edge directions. Algorithms such as Kosaraju's and Tarjan's find SCCs in $O(V + E)$ time. An important distinction is that simply ignoring edge directions finds weakly connected components, not SCCs.

8 Detecting Cycles

Cycle detection depends on whether the graph is directed or undirected. In an undirected graph, DFS detects a cycle if it reaches an already visited neighbor other than the vertex it came from. In a directed graph, DFS detects a cycle if it reaches a vertex currently on the recursion stack. A visited vertex that has already finished processing does not automatically imply a cycle.

```
import java.util.*;

class UndirectedCycle {
    static boolean dfs(int u, int parent,
                       List<List<Integer>> graph,
                       boolean[] visited) {
        visited[u] = true;
        for (int v : graph.get(u)) {
            if (!visited[v]) {
                if (dfs(v, u, graph, visited)) return true;
            } else if (v != parent) {
                return true;
            }
        }
        return false;
    }
    static boolean hasCycle(List<List<Integer>> graph) {
        boolean[] visited = new boolean[graph.size()];
        for (int i = 0; i < graph.size(); i++) {
            if (!visited[i] && dfs(i, -1, graph, visited)) {
                return true;
            }
        }
        return false;
    }
}
```

Note: This particular parent-check implementation assumes a simple undirected graph without parallel edges. For multigraphs, track edge identities instead.

9 Which Algorithm Should We Use?

Problem	Algorithm	Typical time
Unweighted shortest path	BFS	$O(V + E)$
Weighted shortest path, no negative edges	Dijkstra	$O((V + E) \log V)$
Shortest path with negative edges	Bellman Ford	$O(VE)$
Minimum spanning tree	Kruskal	$O(E \log E)$
Minimum spanning tree	Prim	$O(E \log V)$
Prerequisite ordering in a DAG	Topological sort	$O(V + E)$
Connected components	BFS or DFS	$O(V + E)$
Directed SCCs	Kosaraju or Tarjan	$O(V + E)$

10 Common Mistakes to Avoid

1. **Using BFS for arbitrary weighted shortest paths.** BFS minimizes the number of edges, not the sum of unequal edge weights.
2. **Using Dijkstra with negative weights.** The standard correctness argument requires nonnegative weights.

3. **Confusing an MST with a shortest path tree.** An MST minimizes the total weight needed to connect all vertices. A shortest path tree minimizes distances from a chosen source.
4. **Forgetting to add both directions in an undirected adjacency list.** Every undirected edge must be represented from each endpoint.
5. **Forgetting disconnected vertices.** Start a new BFS or DFS for every unvisited vertex when counting components.
6. **Attempting topological sorting on a directed cycle.** Topological ordering is possible only for DAGs.
7. **Treating every visited neighbor as a cycle in an undirected DFS.** The edge back to the parent must be ignored.
8. **Using `int` for potentially large distance sums.** Use `long` when path weights may add up to a large value.

11 Practice Questions

1. A graph has edges A to B (5), A to C (2), C to B (1), and B to D (3). What is the shortest distance from A to D? Which path gives it?
2. Why does an MST of a connected graph with 12 vertices have exactly 11 edges?
3. In Kruskal's algorithm, why do we reject an edge whose endpoints are already in the same Union Find set?
4. A prerequisite graph contains A to B, B to C, and C to A. Can it have a topological order? Explain.
5. A graph has two separate groups of connected vertices and one isolated vertex. How many connected components does it have?
6. How does the purpose of a priority queue differ in Dijkstra and Prim?
7. What happens if Bellman Ford can still relax an edge after $V - 1$ full passes?

11.1 Answers

1. A, C, B, D, total cost $2 + 1 + 3 = 6$.
2. Every tree on V vertices has $V - 1$ edges, so 11 edges.
3. The endpoints are already connected, so adding the edge would create a cycle.
4. No. The directed cycle makes it impossible to place each prerequisite before its successor.
5. Three components.
6. Dijkstra prioritizes the smallest known distance from the source; Prim prioritizes the cheapest edge crossing from the current tree to an unvisited vertex.
7. There is a negative cycle reachable from the source.